NetworkArena A1 — Design Card

Team 25 · link-state · 20/20 practice · worst recovery **196 ms** of 1000 · loops 0 P1 / 4 of 15 P2 · **FRONT: the design**

DESIGN IN ONE LINE

Every switch shouts **HELLO** down every cable; hearing one teaches you who is on that port, and the *timestamp* is how you notice they went silent. Each switch describes only *itself* in an **LSA** and floods it; sequence numbers stop the flood looping. Everyone's LSAs = the map → **BFS** → install a next-hop port per /24. **Family: OSPF.**

THE TWO HALVES

Data plane touches every packet: dumb, fast, table lookups only. Cannot think. Anything it can't match it **punts**.

Control plane (my code) is the brain: remembers, decides, fills tables. Slow; **only ever learns from a punt or a timer**.

The brain never sees the fast path. That split is the assignment.

TIMING — AND WHY

Const	Value	Why
TICK	50 ms	HELLO cadence
DEAD	150 ms	**3 missed HELLOs** — one lost packet must not flap a route
REFRESH	1 s	safety net if a flood is lost

```
detect silence  ≤150 ms
flood network    ~1 ms   (diam 2, 100µs)
BFS + install    ~0 ms
─────────────────────
measured worst  196 ms   budget 1000
```

*500 ms hellos → 1500 ms detect → C4 fails. DEAD below TICK → neighbours expire between hellos; tested, delivery fell to **5.87%**.*

WIRE FORMAT — PROTO 89, TTL 1

```
HELLO [1][sender:u32]                5 B
LSA   [2][origin:u32][seq:u64]
      [prefix:u32][n:u8][n×id]  18+4n B
```

Why TTL 1 if LSAs cross the network? Each hop is a *fresh injection by that switch's controller*, not a forward — consumed, understood, re-emitted. So flooding needs no TTL budget and a stale LSA can't circulate.

Why proto 89? Customer traffic is proto 0. One byte cleanly separates mine from theirs.

TABLES & PIPELINE

```
stage 0  load ip_proto → T_PROTO  exact
         proto 89 → Punt(Custom 89)
stage 1  load ip_dst  → T_ROUTE   LPM
         /24 → SetEgress port
```

Without the proto-89 punt entry a HELLO falls through to the route lookup and is forwarded as ordinary traffic — **no switch ever learns anything**.

T_ROUTE starts **empty**. That's why the first customer packet on every switch punts — and that punt is how I discover my own customer.

WHAT EACH SWITCH REMEMBERS

```
neighbor_on_port  port → switch id
port_of_neighbor  switch id → port
last_heard        port → t of last HELLO
app_port          the customer-facing port
my_prefix         my customer's /24
lsdb              switch id → newest LSA
installed         prefix → port I pushed
self_changed      re-originate my LSA?
```

Both directions of the neighbour map are kept: HELLO arrives knowing the *port*; BFS answers with a *switch*. Each lookup needs the opposite key.

FIVE MECHANISMS

- **Neighbour discovery** — wire gives the port, payload gives the sender id. Neither half is enough alone.
- **Prefix discovery** — a port that *never answers a HELLO* faces a customer. Its punted traffic's `ip_src & 0xFFFFFF00` is my /24. Self-correcting: a later HELLO there undoes the guess.
- **Flooding** — `seq ≤ stored` → drop. That one check is the entire loop prevention. Re-flood out every port *except* the ingress.
- **BFS** — links identical (100 µs, 1 Gbps) so shortest = fewest hops; no priority queue needed.
- **Failure detection** — no alert exists. `now − last_heard ≥ 150 ms` → drop neighbour → re-originate LSA → reflood → recompute.

THE ROUTING ALGORITHM

```
first_hops():
  seen  = {me}
  queue = []
  for (nb,port) in alive_neighbours():   # port order
      first[nb] = nb ; seen+=nb ; queue+=nb
  while queue:
      u = pop_front(queue)
      for v in lsdb[u].neighbours:
          if v in seen:      continue
          if not edge(u,v):  continue   # BOTH claim
          first[v] = first[u]
          seen += v ; queue += v

recompute():
  want = {my_prefix: app_port}
  for (sw,lsa) in lsdb, sw != me, lsa.prefix != 0:
      via = first[sw]
      want[lsa.prefix] = port_of_neighbor[via]
  emit only the DIFFS vs `installed`
  withdraw prefixes no longer reachable
```

Only diffs are emitted — re-sending unchanged entries would burn the 4096-byte action budget for nothing.

TWO RULES AGAINST BLACK HOLES

Bidirectional edge. Believe u–v only if *both* u and v claim it. After a failure one side notices first; for a few ms their LSAs disagree, and trusting a one-sided edge routes traffic into a hole.

Tie-break = lowest port. `alive_neighbors()` iterates a `BTreeMap` keyed by port, so BFS seeds the lowest-numbered port first and it claims the destination. **Deterministic, never random** — a `HashMap` could route differently between runs.

THE TRAP THAT COST 15 RUNS

```
install → entries.push()    APPENDS
lookup  → returns FIRST match
sort    → STABLE; all /24, prio 0
⇒ the original entry never moves
```

An update is **accepted, logged, then silently ignored**. Part 1 passed (converge once, never update); every *down* failed; every *up* looked perfect because the stale entry became valid again. **Fix: `delete_entry` then `install_route`.**

*The tell was **exactly 6/12** recovered. Exactly half = a whole category broken, never a timing problem. Changing constants changed nothing, which ruled out detection entirely.*

REPORT CARD

C1 delivery ≥ **99.9%** P1 / **98.0%** P2. Unique delivered ÷ unique sent. **Misdelivery = a loss.**

C2 reachability — every app pair delivers ≥1 packet. One dead pair = fail.

C3 loops / TTL deaths. **Won't fail you**; burns capacity, which A2 grades.

C4 recovery < **1000 ms**, P2 only. **Per event, never averaged.** Down *and* up both scored.

PASS = C1 ∧ C2 ∧ (C4 pass or skipped). No partial credit inside a run.

*Window: starts at **200 ms** warmup, stops **100 ms** before the end. Early loss is free. **Congestion never drops packets here** — so every lost packet is my program's fault.*

READING A RECOVERY TIMELINE

```
#   t ms  link  event  silence  back   cost
1   2110  2–4   DOWN    +0 ms  +190ms   20  ok
2   4428  2–4   UP        –  never lost  0  ok
3   6865  1–4   DOWN   +15 ms  +135ms   11  ok
```

silence began = when HELLOs actually stopped (may lag the cut). **traffic back** = when losses stopped, not when routes converged. **cost** = packets lost for that event.

"never lost one" on UP = the link returned and was reused without dropping anything.

*Detection from the cut = 150 ms − (time since last HELLO), rounded up to the next 50 ms tick → the **130–190 ms spread**, not one fixed number. That's why #1 is 190 and #3 is 135.*

READING A SIMLOG LINE

```
2300678520 ns  S4  delete  t2  entry id=167825920
2300679288 ns  S4  install t2  key=0xa00d200/24
                    -> SetEgress { port: PortIdW(2) }
```

ns ÷ 1e6 = ms (so 2300678520 ns = **2300.7 ms**). `t2` = T_ROUTE. entry id = the prefix as an integer. **delete immediately followed by install of the same key** = my route update, working as designed.

TRACING A PATH

```
dest 10.0.210.1
1 mask /24    10.0.210.0
2 to hex      0x0a00d200
3 find row    -> port 2
4 PORT→NEIGHBOUR       ← never skip
5 at that switch, repeat from 1
6 port 100 = its app = delivered
```

Hex: `10.0.X.0` → `0x0a00`+hex(X)+`00`. X÷16 = 1st digit, remainder = 2nd; a=10 … f=15.
99→63 · 108→6c · 186→ba · 210→d2 · 221→dd · 226→e2

The table gives a PORT, not a destination. Both my quiz misses were skipping step 4.

PORT NUMBERING

Ports are handed out **per switch, in link-declaration order**, counting from 0. Walk the link list; each time a switch is named it takes its next number. **The two ends of one cable usually differ.** Port 100 = the customer app.

```
link 0: 5–3   S5→p0   S3→p0
link 1: 3–1   S3→p1   S1→p0
link 2: 3–2   S3→p2   S2→p0
link 3: 0–5   S0→p0   S5→p1
⇒ S3: p0=S5 p1=S1 p2=S2
```

Sanity check any map you derive: a switch's own prefix must point at port 100, and each direct neighbour should sit on its own port.

WORKED EXAMPLE — MY RUN

Link **1-4** dies at 6865 ms. S1 ports: p0=S3 p1=S2 p2=S4.

```
before          after (7002 ms)
10.0.108 → p2   10.0.108 → p0
10.0.221 → p2   10.0.221 → p1
```

Only the two routes whose first hop was **p2 (=S4)** moved. `10.0.108` (S0) went 2 hops → **3 hops** (S1→S3→S5→S0) because no 2-hop path survives. `10.0.221` (S4) took p1 (=S2) at 2 hops, strictly shorter than via S3 — **no tie to break**. The other one *was* a tie (both 3 hops), so the lower port won.

CONVERGENCE TIME — MAX ESTIMATES

```
TICK=50  DEAD=150  L=link latency  D=diameter
cold start   ≤ TICK + D·L          = 50 + D·L
link DOWN    ≤ DEAD+TICK + (D+1)·L = 200 + (D+1)·L
link UP      ≤ 2·TICK + (D+1)·L    = 100 + (D+1)·L
```

Why: floods are relayed *instantly* (1 L per hop). Only detection and sending my *own* LSA wait for a tick. t=0 LSAs go nowhere (no neighbours yet) → real LSAs at 50 ms. DOWN = 150 ms silence + up to 1 tick; UP = next HELLO (≤1 tick) + next LSA tick.

Measured (L=0.1 ms): cold 50.28–50.76 · DOWN 102–**200.0** (min ≈ DEAD−TICK=100) · UP 34–**100.3**. UP loses no packets.

Only D and L matter — not switch count, links or load. If L ≥ 50 ms, cold start waits for the first tick after L. Add ~0.03 ms/hop processing; round up slightly.

"WHAT WOULD HAPPEN IF…" — 4 STEPS

```
1 WHO NOTICES?  only the 2 switches on
                that link. Nobody else can.
2 WHEN?         ≤150 ms silence,
                +≤50 ms to the next tick
3 WHAT SPREADS? each endpoint re-originates
                its OWN LSA (seq+1), floods;
                ~1 ms to reach everyone
4 WHAT CHANGES? only routes whose first hop
                crossed that link
```

A link comes back: HELLOs resume → neighbour re-added → `self_changed` → reflood → BFS finds the shorter path again → routes move back. Same 4 steps.

Two links die at once: both detected independently; BFS just runs on a sparser graph. Only fails if the network partitions — the handout promises it won't.

A switch dies: every neighbour sees silence; its LSA ages out of use because no edge stays bidirectional. Its prefix becomes unreachable and I *withdraw* it rather than black-hole it.

PUNT REASONS & LIMITS

```
NoRoute       no T_ROUTE match  (customer)
TtlExpired    hop limit hit
Custom(89)    my own HELLO/LSA
```

`fuel`	4,000,000 instr / call → trap, program dead for the run
`actions`	4,096 B / return (~150 installs) → call rejected
`tables`	16 max, 1,024 entries each

A dead program keeps forwarding on its last routes — from outside it looks exactly like a routing bug.

ADVERSARY & PATCHES

- **Forge a HELLO** with someone else's id → I install a false neighbour. *Patch: sign/authenticate HELLOs.*
- **Replay an LSA** with a huge seq → accepted, and the real origin can't correct it until it exceeds that number. *Patch: bound seq jumps; age entries out.*
- **Lossy, not dead** — if some HELLOs get through I never declare it down and keep using a bad link. *Patch: track loss rate, not just silence.* (Explicitly A2's territory.)
- **Flood storm** — force constant topology churn and my control traffic grows. *Patch: rate-limit re-origination.*
- **Micro-loops** — 0 in P1; **4/15** P2 runs (2/4/19/8 pkts). S5 notices the dead link first, reroutes via S6; S6 hasn't got the new LSA yet and points back → bounce till TTL. Transient; real OSPF has it. *Patch: ordered FIB updates.*

THINGS I'D GET ASKED AND CAN ANSWER

- **Why BFS not Dijkstra?** Every link is 100 µs / 1 Gbps — equal weights, so fewest hops *is* shortest. Dijkstra's queue would be wasted work.
- **Why does a switch flood only its OWN LSA?** Nobody else can describe my adjacency truthfully; and it keeps origination rate bounded.
- **Why 1 s refresh if changes reflood instantly?** A lost flood would otherwise leave someone stale forever.
- **Why is the first packet always lost?** T_ROUTE starts empty; the punt is how I learn. That's inside the 200 ms warmup, so it isn't scored.